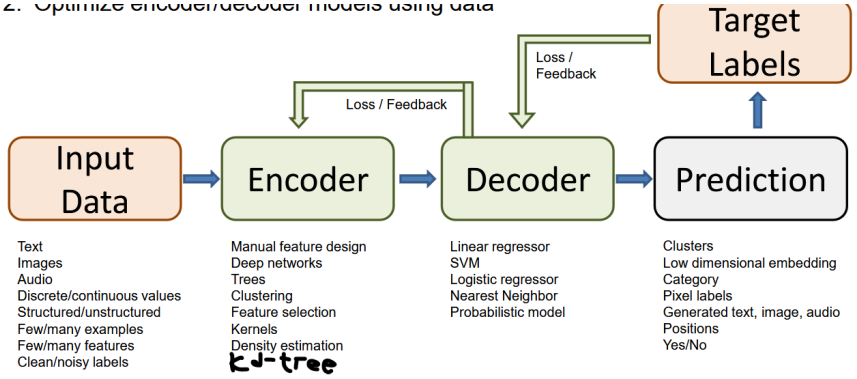
# ML

1. The whole machine learning problem
    1. Data preparation
        1. Collect and curate data
        2. Annotate the data (for supervised problems)
        3. Split your data into train, val, test sets
    2. Algorithm and model development
        1. Design methods to extract features from the data
        2. Design machine learning model and identify key parameters and loss
        3. Train, select params, and evaluate your designs using the val set
    3. Final evaluation using test set
    4. Integrate into your application
2. Optimize encoder/decoder models using data




3. Supervised learning, regression
    1. $f(X) = P(Y|X)$
4. dataset $X$, marginal distribution, $\mu(x) = P(X)$
5. defined so called regression function, $\eta(x) = E[Y|X = x]$, read: expectation of the prediction $Y$ given $X = x$.
6. In the special case of classification, the regression function can be rewritten as $\eta(x) = 0 \cdot P(Y = 0, X = x) + 1 \cdot P(Y = 1|X = x) = P(Y = 1|X = x)$, but this is just regression! or being reduced to a regression problem.
7. If the expectations of the predictions, $\eta(x)$ is close to 0.5, i.e. if $\eta$ is close to 1, then classifying $x$ is easy if $\eta$ is close to $0.5$, then classifying $x$ is difficult.
8. The probability distribution $P(X, Y)$ is uniquely determined by $\mu$ and $\eta$.
9. Intuition (discrete case): We can rewrite $P(X = x, Y = 1) = P(Y = 1|X = x)P(X = x) = \eta(x)\mu(x)$ and similarly, $P(X = x, Y = 0) = P(Y = 0|X = x)P(X = x) = (1 - \eta(x))\mu(x)$
10. So we can express the probability of any event $(x, y)$ in terms of $\eta$ and $\mu$.
11. For formal proof for the general case: see the book of Devorye, Gyorfi, Lugosi, first pages.
12. Explicit form of Bayes classifier under 0-1 loss.
    1. Note: now assume the classifier $f(X) = P(Y|X = x)$ gives either 0 or 1 or $\{0, 1\}$.
    2. The risk of a classifier under the 0-1 loss counts how often the classifier fails, that is $R(f) = E(l(X, Y, f(X))) = E(\mathbf{1}_{f(X)\neq Y}) = P(f(X) \neq Y)$.
    3. The bayes classifier $f^*$ was defined as the classifier that minimizes the **true** risk. This is a hypothetical classifier function.
    4. $$f^0(x) \to \begin{cases} 1, \text{if } \eta(x) \geq 0.5 \\ 0, \text{otherwise} \end{cases}$$
    5. basically 0.5 is an optimal threshold, if set higher than 0.5 you get FN if set lower, you get FP.
    6. $f^0$ is the Bayes classifier. The theorem shows that $f^0 = f^*$ (why?). Consequence: in the particular case of classification with the 0-1 loss, we have an explicit formula for the Bayes Classifier. In practice we do not know $f^*$ why?
    7. Theorem: $f^0$ is the Bayes classifier. Consider classification with 0-1 loss. let $f : x \to \{0, 1\}$ be any measurable classifier and $f^0$ the classifier defined above. Then $R(f) \geq R(f^0)$. Then why don't we just find the Bayes classifier for all problems and go home?
    8. Answer: we can only compute the Bayes Classifier if we know the underlying probability distribution, namely we need to evaluate the function $\eta$, we can evaluate $\eta$ only if we know the underlying distribution, but we cannot get the underlying

distribution if only based on a limited set of data.

9. Proof:
10. Step 1: consider any fixed classifier $f : x \to \{0, 1\}$ and compute its error probability at some fixed point $x$: loss:

$$\begin{aligned} P(f(x) \neq Y|X = x) &= 1 - P(f(x) == Y|X = x) \\ &= 1 - P(f(x) = 1, Y = 1|X = x) - P(f(x) = 0, Y = 0|X = x) \\ &= 1 - 1_{f(x)=1}P(Y = 1|X = x) - 1_{f(x)=0}P(f(x) = 0|X = x) \\ &= 1 - 1_{f(x)=1}\eta(x) - 1_{f(x)=0}(1 - \eta(x)) \end{aligned}$$

11. Step 2: Now compare the pointwise error of any particular classifier $f$ to the one of $f^0$, $P(f(X) \neq Y|X = x) - P(f^0(X) \neq Y|X = x)$.
12. Plugging in the equation from step 1, and using the property $1_{x=0} = 1 - 1_{x=1}$ or $1_{x<0} = 1 - 1_{x\geq 1}$,

$$\begin{aligned} P(f(X) \neq Y|X = x) - P(f^0(X) \neq Y|X = x) = \\ 1 - 1_{f(x)=1}\eta(x) - (1 - 1_{f(x)=1})(1 - \eta(x)) - 1 + 1_{f^0(x)=1}\eta(x) + (1 - 1_{f^0(x)=1})(1 - \eta(x)) = \\ (1_{f^0(x)=1} - 1_{f(x)=1})(2\eta(x) - 1) \end{aligned}$$

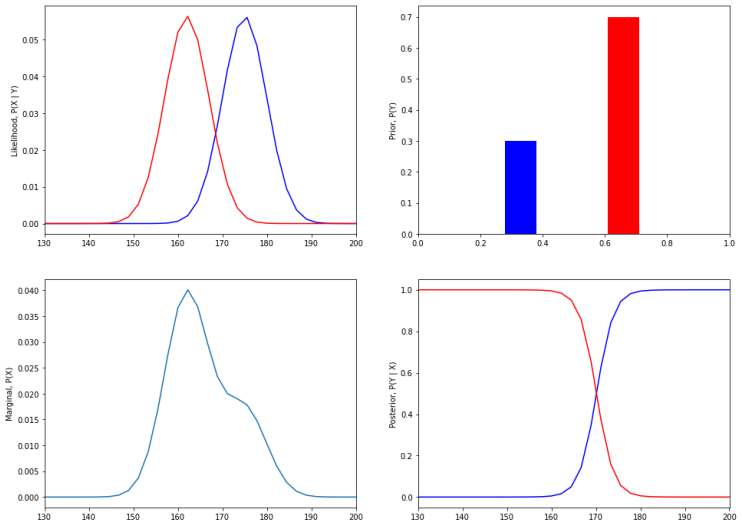
13. if $1_{f^0(x)=1} = 1 \Rightarrow \eta(x) \geq 0.5$ then the first term $(1_{f^0(x)=1} - 1_{f(x)=1})$ is $+$, the second term $(2\eta(x) - 1)$ is $+$.
14. if $1_{f^0(x)=1} = 0 \Rightarrow \eta(x) < 0.5$ then the first term $(1_{f^0(x)=1} - 1_{f(x)=1})$ is $-$, the second term $(2\eta(x) - 1)$ is $-$.
15. Hence, loss=$(2\eta(x) - 1)(1_{f^0(x)=1} - 1_{f(x)=1}) \geq 0$.
16. Step 3: we have seen that for all fixed values of $x$, $P(f(x) \neq Y|X = x) \geq P(f^0(x) \neq Y|X = x)$ where $f^0$ is the Bayes classifier. Because this holds for any individual value of $x$, it also holds in expectation over all $x$. This implies $R(f) \geq R(f^0)$.

13. Summary:
    1. If we work with 0-1 loss, one don't need to 'learn' the optimal classifier if we know the function $\eta(x)$. However in real life, we don't know the function $\eta(x)$.
    2. For may other loss functions one can also explicitly compute the optimal classifier, one such example is regression with squared loss.
    3. Problem in practice is we don't know the underlying prior distribution (population data) $\mu(x)$ and the regression function $\eta(x)$.
14. Note: Linear regression
    1. Why not just use matrix inversion? because matrix inversion scales badly with number of features d. With n rows and d features compute is O(d^3 + nd^2) and overhead space is O(d^2). Iterative (gradient descent) methods are better for larger datasets. For small d matrix inversion scales just as well as gradient descent.
15. Empirical Risk Minimization (ERM)
    1. $R_n(f) = \frac{1}{n}\sum_{i=1}^{n} l(X_i = x_i, Y_i = y_i, f(X = x_i))$
    2. Define a set $\mathcal{F}$ of functions from $X \to Y$.
    3. Within these functions, choose one that has the smallest empirical risk.
        1. $f_n = \arg\min_{f\in\mathcal{F}} R_n(f)$
        2. (if minimum does not exist, only infimum exists, then it will be that converges to a function $f_n$. The function sequence classifier functions may not even be unique, but in practice it does not pose any problems).
    4. Estimation vs approximation (model) error
    5. Underfitting means $\mathcal{F}$ is too small:
        1. small estimation error.
        2. large model error (approximation error).
    6. Overfitting means $\mathcal{F}$ is too large:
        1. large estimation error.
        2. small model error (approximation error).
    7. Bias-variance trade-off
        1. Bias term: how much difference between $f_n$ and $f^*$ (same flavour as model error).
        2. Variance term: the variance of the RV $f_n$ (estimation error).
    8. ERM is a straightforward learning principle.
    9. The key to success/failure of ERM is to choose a 'good' function class $\mathcal{F}$.
    10. Finding the minimizer of the 0-1 loss can be challenging as it is often NP-hard. In practice, we use convex relaxation of the 0-1 loss function, see below.
    11. Approach:

    1. Define regularized risk: $R_{reg,n}(f) = R_n(f) + \lambda\Omega(f)$. Here $\lambda > 0$ is called a regularizer constant.
    2. Then choose $f \in \mathcal{F}$ to minimize the regularized risk.
    3. Regularizer, $\Omega : \mathcal{F} \to \mathrm{R}_{>0}$, that measures how 'complex' a function is,
    4. Examples:
        1. $\mathcal{F} = \text{polynomials}$, $\Omega f = \text{degree of the polynomial f}$.
        2. $\mathcal{F} = \text{differential functions}$, $\Omega(F) = \text{maximal slope}$.
    5. if $\lambda$ very small: overfitting, high bias error.
    6. if $\lambda$ very large: underfitting, high variance error.
16. when we talk about a distribution
    1. model
    2. parameter
    3. random variable
17. distribution models (based on study order):
    1. Bernoulli, RV: $X = \begin{cases} 1, & \text{with probability p} \\ 0, & \text{with probability q=1-p} \end{cases}$, example: coin tossing $\to$ multinoulli (categorical), RV: 1 to $k$, example: dice casting, word drawing. Softmax function is often used to predict the probabilities associated with multinoulli distribution.
    2. Binomial, RV: the results of n i.i.d. Bernoulli distributions, example: for coin tossing, getting exactly $k$ successes in $n$ independent trials $\to$ multinomial: the results of n i.i.d. categorical distribution, example: cast a dice n times.
    3. Uniform, RV: a **continuous** real number $\to$ vector RV, like a elevated 2D plane in a 3D plot.
    4. Normal/Gaussian, RV: a **continuous** real number, bell shaped, with the central point is the most probable value $\to$ vector RV, like a sombrero hat in a 3D plot.
    5. Poisson, RV: number of events in a fixed interval of time (or space).
    6. Geometric, RV: the amt of independent failures $k$ until the first success, each with success probability $p \to$ Exponentials, RV: continuous analogue of geometric distribution, the amt of time until a next event occurs (success).
    7. Gamma, RV: the wait time until the $k$-th event occurs.
    8. Chi-squared, RV: sum of squared of independent standard normal RVs.
    9. Beta, RV: continuous value between 0 and 1, i.e. $[0, 1] \in \mathrm{R}$.
18. Bayesian decision rule
    1. Given the height of a person we want to predict the gender.
    2. RV: Y: male or female, RV: X: height.
    3. Code:

```
# -*- coding: utf-8 -*-
"""
Created on Sat Apr 11 20:17:07 2026

@author: reina
"""

import numpy as np
import matplotlib.pyplot as plt

# probability density function or likelihood
def gaussian_pdf(x, mu, var):
a = 1 / np.sqrt(2 * np.pi * var)
b = - (x - mu)**2 / var
return a * np.exp(b)

# RV: X: height
# RV: Y: male or female
# want to predict P(Y | X)

# Define likelihood
x = np.linspace(0, 220, num=100)
mu1 = 175; var1 = 50
mu2 = 162; var2 = 50
likelihood1 = gaussian_pdf(x, mu1, var1)
likelihood2 = gaussian_pdf(x, mu2, var2)

# Define priors
prior1 = 0.3
prior2 = 1 - prior1

# Compute marginal
marginal = likelihood1 * prior1 + likelihood2 * prior2

# Compute posteriors
posterior1 = likelihood1 * prior1 / marginal
```

```
posterior2 = likelihood2 * prior2 / marginal

fig = plt.figure(figsize=(16, 12))

# Plot likelihood
ax1 = fig.add_subplot(221)
ax1.plot(x, likelihood1, color='b')
ax1.plot(x, likelihood2, color='r')
ax1.set_xlim(130, 200)
ax1.set_ylabel('Likelihood, P(X | Y)')

# Plot priors
ax2 = fig.add_subplot(222)
ax2.bar(0.33, prior1, width=0.1, color='b')
ax2.bar(0.66, prior2, width=0.1, color='r')
ax2.set_xlim(0, 1)
ax2.set_ylabel('Prior, P(Y)')

# Plot marginal
ax3 = fig.add_subplot(223)
ax3.plot(x, marginal)
ax3.set_xlim(130, 200)
ax3.set_ylabel('Marginal, P(X)')

# Plot posterior
ax4 = fig.add_subplot(224)
ax4.plot(x, posterior1, color='b')
ax4.plot(x, posterior2, color='r')
ax4.set_xlim(130, 200)
ax4.set_ylabel('Posterior, P(Y | X)')
```




## Logistic regression

1. Steps:
    1. compute distance to the hyperplane via dot product, $d = w^T x$, since $w$ is the vector normal to the hyperplane.
    2. pass the distance through a sigmoid function that outputs $0$ or $1$.
    3. compute loss using cross-entropy loss or binary cross entropy loss.
2. Although the sigmoid function is nonlinear, the classification itself is a linear operation since it has a linear hyperplane.
3. $P(y = 1|X) = \frac{1}{1+e^{-(w_0+w_1 x)}}$, $P(y = -1|X) = \frac{1}{1+e^{(w_0+w_1 x)}}$.
4. $\sigma = \frac{1}{1+e^{-(\vec{w}^T \vec{x}_i)}}$, $y_i = \sigma(\vec{w}^T \vec{x}_i)$.
5. $$y_i = \begin{cases} 1, \text{if } \sigma(\vec{w}^T \vec{x}_i) \geq 0.5 \\ -1, \text{if } \sigma(\vec{w}^T \vec{x}_i) < 0.5 \end{cases}$$
6. $\frac{1}{1+e^{-\vec{w}^T \vec{x}_i}} > 0.5 = \frac{1}{1+1} \Rightarrow e^{-\vec{w}^T \vec{x}_i} < 1 \Rightarrow e^{-\vec{w}^T \vec{x}_i} < e^0 \Rightarrow -\vec{w}^T \vec{x}_i < 0.$
7. $-\vec{w}^T \vec{x}_i < 0, y_i = 1 \Rightarrow y_i(\vec{w}^T \vec{x}_i) > 0.$
8. similarly, for negative labels: $-\vec{w}_i^T \vec{x} > 0, y_i = -1 \Rightarrow y_i(\vec{w}^T \vec{x}_i) > 0$. That means the condition that should be checked is whether $y_i(\vec{w}^T \vec{x}_i) > 0$ is satisfied for each point $x$. Otherwise, update $w$ using SGD until the condition is satisfied.

## K-means:

1. Special case of EM, special case of Autoencoder
2. discrete distribution

3. K-means:
    1. randomly place K centroids, where K is the number of clusters we want.
    2. these centroids act like gravity pulling closer points into their orbit.
    3. the algorithm calculates the distance of each data point to the centroids assigning data points to the closest one forming clusters.
    4. now the centroid shift to the average position of all the points in their cluster.
    5. the process repeat until the centroids stop moving.
4. Simple and efficient, but need to specify K.
5. Code: k-means.py

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Apr 13 21:27:53 2026

@author: reina
"""

import numpy as np


def k_means_clustering(X, k, max_iter=100):
    # Step 1: Initialize cluster centroids randomly
    centroids = X[np.random.choice(range(len(X)), k, replace=False)]

    # Step 2: Repeat until convergence or max_iter reached
    for _ in range(max_iter):
        # Step 2a: Assignment - assign each data point to the nearest cluster centroid
        # (N, D) - (k, 1, D) = (k, N, D) -sum(axis=2)-> (k, N)
        distances = np.sqrt(((X - centroids[:, np.newaxis])**2).sum(axis=2))
        # (N,)
        cluster_assignments = np.argmin(distances, axis=0)
        # Step 2b: Update - compute new centroids basde on the mean of the data points in each cluster
        new_centroids = np.array([X[cluster_assignments == i].mean(axis=0) for i in range(k)])

        # Check for convergence
        if np.all(centroids == new_centroids):
            break

        centroids = new_centroids

    return centroids, cluster_assignments


# def k_means_clustering_naive(X, k, max_iter=100):
#     centroids = X[np.random.choice(range(len(X)), k, replace=False), :]
#     N, D = X.shape
#     for _ in range(max_iter):

#         cluster_assignments = []
#         for i in range(N):
#             distances = []
#             for j in range(k):
#                 dist = np.sqrt(np.sum((X[i, :] - centroids[j])**2))
#                 distances.append(dist)
#             distances = np.array(distances)

#             cluster = np.argmin(distances, axis=0)
#             cluster_assignments.append(cluster)

#         cluster_assignments = np.array(cluster_assignments)

#         new_centroids = np.array([X[cluster_assignments == m].mean(axis=0) for m in range(k)])

#         if np.all(new_centroids == centroids):
#             break

#         centroids = new_centroids

#     return centroids, cluster_assignments


if __name__ == '__main__':
    # Example usage
    X = np.array([[2, 3], [3, 4], [5, 4], [7, 2], [8, 1], [9, 0]])
    k = 2

    centroids, cluster_assignments = k_means_clustering(X, k)
    # centroids, cluster_assignments = k_means_clustering_naive(X, k)
    print("Final centroids:")
    print(centroids)
    print("Cluster assignments:")
    print(cluster_assignments)
```

References:

1. CS441 UIUC, Derek Hoeim, 2025.
2. Statistical Machine Learning, Tubingen, 2024.